

I/O from Python to the operating system

The call you write is not the call the operating system runs. `fflush()` flushes only the C library's buffer, and the kernel buffer needs a second call [S3]. Four boundaries sit below your code: app to library, library to kernel, kernel to device, and device to medium. A different tool watches each of the first three [V10] [S1] [V30]. No tool here watches the fourth [V5].

Ondřej (Ondra) Krajíček

me@ondrejkrasicek.com · www.linkedin.com/in/OndrejKrajicek

Built 2026-09-15 15:59 UTC

Generated by the agentic vault — an autonomous research and authoring harness. Every factual claim was gated against a cited source registry before this document was produced.

[S1] `strace(1)` — Linux man-pages. <https://man7.org/linux/man-pages/man1/strace.1.html> [S3] `fflush(3)` — Linux man-pages. <https://man7.org/linux/man-pages/man3/fflush.3.html>

Roadmap

1	What does one byte cost?	A stopwatch.
2	Where does the difference come from?	A tracer.
3	It is not one buffer — it is a stack	A diagram.
4	The same write, at three levels	A program.
5	Two operating systems, side by side	Two traces.
6	What we have not looked at	One open question.

Takeaways

- | | | |
|----------|--|---|
| 1 | Adjacency decides the call count | The saving is in calls, not work, and only for adjacent bytes. A seek empties the buffer. |
| 2 | A library call is not a system call | Several levels sit beneath yours [S3]. |
| 3 | The boundary is observable | A tracer shows what your line became [S1]. |
| 4 | The platforms differ by design | Each nominates a different level [V8]. |

SCOPE

What I cover today

- 1 Why two programs differ
- 2 When a buffer helps
- 3 The four levels
- 4 Reading a trace

SCOPE

What I leave out today

5 When a write is safe

6 Memory mapping

7 Operating the tools

8 Asynchronous I/O

PART I

What does one byte cost?

Two programs. The same mebibyte. Predict the ratio before we run them.

What does one byte cost?

Two programs write the same mebibyte to a file. That is 1,048,576 bytes. One uses a buffer. One does not.

```
# A: unbuffered — one call per byte
with open("a.out", "wb", buffering=0) as f:
    for i in range(1024 * 1024):
        f.write(b"x")

# B: buffered — the default
with open("b.out", "wb") as f:
    for i in range(1024 * 1024):
        f.write(b"x")
```

One argument differs. Python documents it: "Pass 0 to switch buffering off (only allowed in binary mode)" [S6].

[S6] Built-in Functions, `open()` — Python documentation. <https://docs.python.org/3/library/functions.html#open>

Two programs, two access patterns

Sequential access

Each write continues where the last one ended.

Random access

Each write follows a move to a new offset.

Why it matters

Lampson advised batch processing in 1983. He set sequential access aside from that advice, with the words "even aside from" [S12].

[S12] Hints for Computer System Design — Lampson (1983), p. 19. <https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/acrobat-17.pdf>

Predict before we run

Write down a ratio. How much slower is the unbuffered program?

1×

nothing worth measuring

10×

an optimisation

100×

a different kind of thing

Writing a number down before the run changes what you do with the result. Taber calls the method predict-observe-explain, and reports that "making a prediction will give students some sense of ownership in the following activity" [S14].

[S14] Predict-Observe-Explain — Taber. <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/>

The measurement runs here, not on a slide

I print no ratio here, on purpose. A benchmark you did not watch me produce is the kind of claim this course teaches you to distrust.

Held fixed

Loop, interpreter, iteration count, bytes, filesystem.

Varied

One argument, `buffering=0` [S6].

Write down

Both times, the Python version, the buffer size.

What it includes

The operating system's own memory for file data, in both runs. Name the cache layer your number includes, because you cannot remove all of them [V19].

Your laptop will not agree with this projector

The buffer size is not a constant. It grew sixteen-fold in Python 3.14.

Python 3.13

The default "was set to 8192 since 16 years ago" [S7].

Python 3.14

"the buffer will typically be 128 kilobytes long" [S6].

Why

Write performance improved "3 to 5 times" over the 8192-byte default [S7]. The reporter measured it. No independent reproduction.

So

Report your version and buffer size with every number.

[S6] Python documentation, `open()`. [S7] CPython issue 117151. <https://github.com/python/cpython/issues/117151>

Where does the difference come from?

One answer, stated immediately, and then the instrument that proves it.

Where does the difference come from?

The difference is the number of times each program crosses into the operating system.

Your theory course defined a system call. Today I count them: each one makes a **crossing** out of your program, "a privilege transition, not a jump" [V15].

A	Unbuffered	One call per byte, so 1,048,576 calls.
B	Buffered	One mebibyte divided by the buffer size.
=	So measure first	At 8 KiB, 128 calls. At 128 KiB, 8 calls.

What one crossing costs

4,972

instructions, one stat call

13,585

cycles for the same call

2,559

L3 lines evicted — your program was running

Soares and Stumm measured this on 2010 hardware [S16]. The counts are not current [V44]. What carries over is the regime distinction, not the numbers. That distinction is a call rate. The lost cache state dominates at one call per 2,000 to 100,000 instructions. The transition cost takes over above that rate [S16].

[S16] FlexSC — Soares and Stumm, OSDI 2010. https://www.usenix.org/legacy/event/osdi10/tech/full_papers/Soares.pdf

strace: the tool that watches the boundary

A tracer runs your program and writes down its system calls [S1].

What it does

It "intercepts and records the system calls made by a process" [S1]. It never samples.

Where it comes from

Paul Kranenburg wrote the original for SunOS. Branko Lankester ported it [S1]. The licence is the Lesser GPL [S30].

HOW TO USE IT

```
strace ./iodemo           # one line per system call [S1]
strace -c ./iodemo        # calls, errors, time per call [S1]
strace -e trace=write ./iodemo # only the writes [S1]
```

It is a Linux tool [S30]. It costs speed [S1]. I use NtTrace in Part V [S10].

[S1] strace(1) man page. <https://man7.org/linux/man-pages/man1/strace.1.html> [S30] strace. <https://strace.io/> [S10] NtTrace. <https://rogerorr.github.io/NtTrace/>

What one trace line says

A trace records the system calls a program made, in order, with arguments and return values.

```
open("/dev/null", O_RDONLY) = 3
```

- 1 The name** Which system call ran [S1].
- 2 The arguments** In parentheses, decoded [S1].
- 3 The return value** After the equals sign [S1].

A trace is not a log. A log carries what the program's author chose to print. A trace carries what the program did.

[S1] `strace(1)` — Linux man-pages, which documents this example line. <https://man7.org/linux/man-pages/man1/strace.1.html>

Count the crossings

The same two programs, run again under a counting tracer.

Command	<code>strace -c -e trace=write python3 a.py</code> [S1]
Program A	One write per byte.
Program B	One write per buffer fill.
First	Print <code>io.DEFAULT_BUFFER_SIZE</code> , then divide [S6].
Two runs	A traced run cannot be timed [V17].

[S1] `strace(1)` — Linux man-pages. <https://man7.org/linux/man-pages/man1/strace.1.html>

The count ratio over-predicts the time

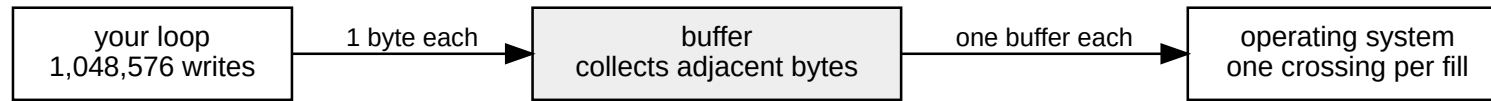
Illustration, not a measurement from this room: a count ratio of 100,000 beside a time ratio of 40 is not a mistake.

$\text{time} \approx \text{crossings} \times \text{fixed cost} + \text{bytes} \times \text{marginal cost [V1]}$

- 1 Two terms** Not a function of bytes alone [V1].
- 2 One term is shared** Both programs write the same mebibyte through the same loop.
- 3 So counts over-predict** Under this model the count ratio exceeds the time ratio.

What a buffer actually does

A buffer collects adjacent bytes and hands them down in one call.



Cost has two terms: the number of crossings, and the number of bytes [V1]. A buffer cuts the first and leaves the second alone, so the saving sits entirely in the call count.

Why a seek gives the saving back

A buffer holds bytes for consecutive positions. Move elsewhere and those bytes cannot stay in it.

- | | | |
|----------|--------------------|---|
| 1 | Constraint | Buffered bytes belong at the old offset. |
| 2 | Consequence | Every seek forces the buffer out. |
| 3 | Result | The buffered run converges on the unbuffered. |

The rule is [buffering is fast when writes are adjacent, and buys little when they are not](#). Lampson gave the batching half in 1983 [S12].

Does the seek itself cost a crossing?

Yes. And a pair of system calls exists to remove it.

Naive

`lseek` then `write` — two crossings for one operation.

Merged

`pread` and `pwrite` "combine[s] file access and seek operations into a single command" [V3].

Measured

The merged pair cut the call count [V3]. For the same data volume the study also reports "a relatively reduced load in terms of overall duration" [V27].

Trust the counts. Do not trust the times.

The tracer stops your program twice per call. That destroys the timing and leaves the counting intact.

EVIDENCE	WHAT IT SAYS	TAKE IT AS
Every call stops	so "the counts are trustworthy and the absolute times are not" [V13]	counts only
Gregg's dd loop	one byte per call, "runs 442× slower under strace" [V17]	his machine
Re-runs	"roughly 37× to 136×", other hardware [V31]	not a bound

The range does not contain Gregg's figure. Each number measures one machine and one workload.

Windows counts too — differently

NtTrace, the Windows tracer in this course, prints no counting summary. Both systems count for you anyway.

PLATFORM	WHERE THE COUNT LIVES	WHAT THE FIELD IS DEFINED AS
Windows	GetProcessIoCounters fills IO_COUNTERS [S24]	WriteOperationCount — "The number of write operations performed" [S23]
Linux	the file /proc/pid/io [S25]	syscw — the "file write" system calls [S25]

One source file, --count on both: Windows counted 1 write operation, the Linux container counted 7 [S37].

[S23] IO_COUNTERS (winnt.h) — Microsoft Learn. https://learn.microsoft.com/en-us/windows/win32/api/winnt/ns-winnt-io_counters [S24]
GetProcessIoCounters (winbase.h) — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/api/winbase/nf-winbase-getprocessiounters> [S25] proc_pid_io(5) — Linux man-pages. https://man7.org/linux/man-pages/man5/proc_pid_io.5.html [S37] Both recorded runs of iodemo, 2026-09-15 — demo/iodemo-c/README.md

Counting on Windows: ask the kernel

strace is a Linux tool [S30]. Windows accounts for a process's I/O anyway, and one documented call reads it back [S24].

```
def write_operations():
    handle = win32api.GetCurrentProcess()
    return win32process.GetProcessIoCounters(handle)["WriteOperationCount"]
before = write_operations()
with open(path, "wb") as f:
    f.write(b"one\n"); f.write(b"two\n"); f.write(b"three\n")
after = write_operations()
```

- **Why not a tracer.** NtTrace prints no summary. The program asks the kernel [S24].
- **What it does.** Reads WriteOperationCount [S23] twice. The difference is what three lines cost.
- **What to expect. One** operation, not three — Python buffered them [S6], and my run measured one [S35].

What the counters will not tell you

Three limits, none visible in the count.

THE LIMIT	WHAT IT DOES TO YOUR COUNT
Not one measurement	Linux counts the <code>write(2)</code> family of system calls [S25]. Microsoft says "write operations performed" [S23].
Operations, not crossings	<code>syscw</code> counts only write operations [S25]. It misses a seek.
Start-up writes count too	The count covers the process, not your loop. Compare two runs.

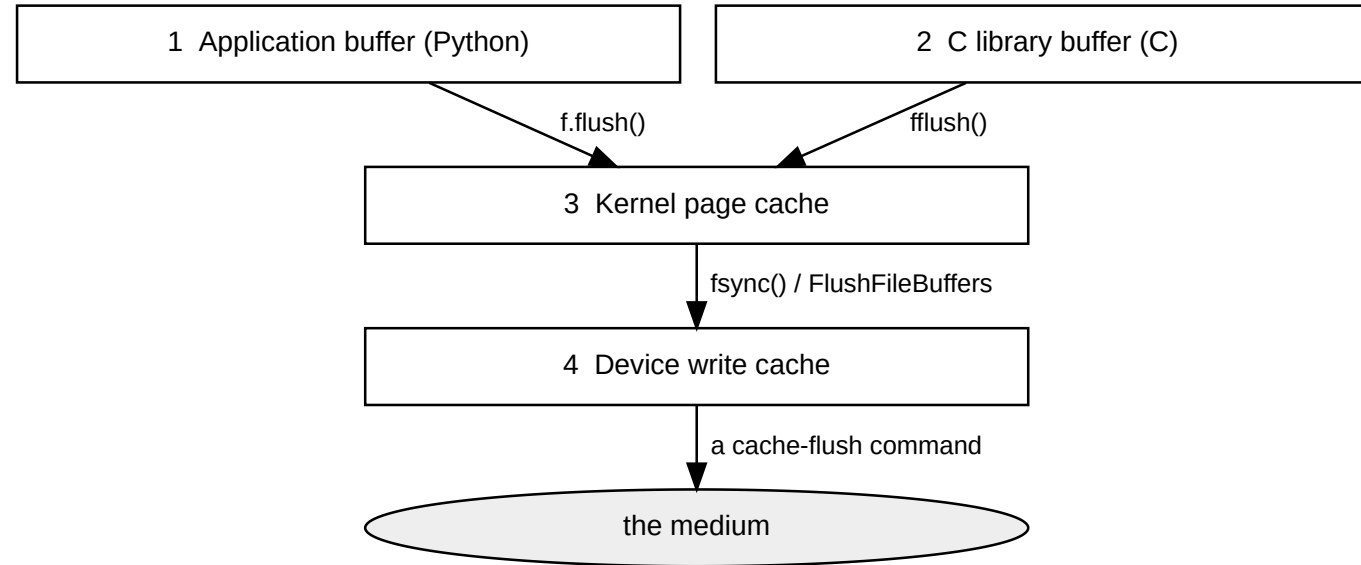
Part V reads a Windows trace I captured [S34]. Equal footing is a course promise, not a per-lecture one.

[S23] `IO_COUNTERS` (`winnt.h`) — Microsoft Learn. https://learn.microsoft.com/en-us/windows/win32/api/winnt/ns-winnt-io_counters [S25] `proc_pid_io(5)` — Linux man-pages. https://man7.org/linux/man-pages/man5/proc_pid_io.5.html [S34] Our NtTrace64 `cmd.exe` capture.

It is not one buffer — it is a stack

We counted the crossings. Now we ask where the bytes actually are.

It is not one buffer — it is a stack



Levels 1 and 2 are alternative routes into level 3 [V6]. Each arrow is one call [S17] [S3] [S13] [V37] [V33].

[S17] `io.IOBase.flush()` — Python documentation. [S3] `fflush(3)` — Linux man-pages. [S13] `fsync` — The Open Group.

Four levels, and who empties each one

LEVEL	WHO OWNS IT	EMPTIED BY
1 Application	the runtime's own buffer [S8]	<code>f.flush()</code> [S17]
2 C library	the <code>stdio</code> buffer [S3]	<code>fflush()</code> [S3]
3 Kernel	the page cache [V32]	<code>fsync()</code> , or <code>FlushFileBuffers()</code> on Windows [V37]
4 Device	the drive's write cache [V20]	a cache-flush command [V33]

The page cache lives in the machine's memory [V32]. It is not the disk's cache — the disk has its own, one level further down [V20].

The kernel also moves data upward. It fetches pages ahead of a sequential reader, so "a sequential `read()` just happens to return faster because the data was already fetched" [V34]. The same levels work in both directions.

To flush is to cross one boundary

The verb means one thing here: to hand a buffer's contents to the next level down.

The C library manual says it plainly: "Note that fflush() flushes only the user-space buffers provided by the C library. To ensure that the data is physically stored on disk the kernel buffers must be flushed too" [S3].



Do not say

Save, commit, persist — all claim durability.



Say

Flush. It names one boundary.

[S3] fflush(3) — Linux man-pages. <https://man7.org/linux/man-pages/man3/fflush.3.html>

Wholesale down, retail up

Every level buys a large block from the level below. It sells small pieces to the level above.

That asymmetry is why every level in this diagram holds a buffer. Each level's "unit of purchase is larger than its unit of sale" [V18].

- | | | |
|----------|-----------------------------|-------------------------------------|
| 1 | Buys in blocks | 4096 bytes on an ext4 file [V7]. |
| 2 | Sells in bytes | Your loop writes one byte. |
| 3 | Same shape elsewhere | Allocators, caches, network stacks. |

One program, one buffer — C and Python take different paths

Each program uses at most one buffer, and not the same one.

```
iodemo.c    fwrite()  →  stdio buffer (level 2)  →  write()  →  page cache  
iodemo.py   f.write() →  BufferedWriter (level 1) →  write()  →  page cache
```

The C program's buffer is the C library's `stdio` [S3] [V7]. Python's is a `BufferedWriter` [S8]. `FileIO` calls `write()` itself, so no `stdio` buffer sits under Python on either platform [S26] [S27] [S28] [V6].

The four levels are the union of both paths: a program uses one of levels 1 and 2, never both. Python documents [128 kilobytes](#) as typical [S6]. On one ext4 file the C library took 4096 bytes, its block size [V7].

[S6] `open()` — Python documentation. <https://docs.python.org/3/library/functions.html#open>

The bottom level acknowledges early too

Level four is the uncomfortable one, and skipping it would make the diagram dishonest.

- 1 The device holds a buffer** "most HDDs and many SSDs acknowledge a write once it lands in an onboard DRAM cache" [V20].
- 2 The standard allows a no-op** On fsync: "a null implementation is permitted" [S13].
- 3 Software cannot check** No trace separates a device that flushed from one that only reported it [V5].

[S13] fsync — The Open Group. <https://pubs.opengroup.org/onlinepubs/9699919799/functions/fsync.html>

Three lines that print nothing

A demonstration. You have the stack now, so predict this one.

```
import os  
print("hello")  
os._exit(0)
```

To a terminal

Does hello appear?

Redirected to a file

What does the file contain?

Write down both answers before I run either one [S14].

[S14] Predict-Observe-Explain — Taber. <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/>

Why the file is empty

The rule sits in the C standard, not in the kernel.

- 1 Mode follows destination**

Output is "fully buffered if and only if the stream can be determined not to refer to an interactive device" [S5].
- 2 A terminal is interactive**

Bytes go out per line.
- 3 A file is not**

Bytes wait.
- 4 The exit skips cleanup**

No write reaches the kernel.

[S5] C99 draft §7.19.3. <https://busybox.net/~landley/c99-draft.html>

What this demonstration does not prove

It proves level one is real. It proves nothing about levels three and four.

"the honest classroom demo is a power-cut, not a kill -9. Killing the process loses only the *runtime's* buffer; the kernel still writes back." [V14]

- | | | |
|----------|-------------------------|---|
| 1 | What died | The process, and the level-one buffer with it. |
| 2 | What survived | Anything already in the page cache [V14]. |
| 3 | Wrong conclusion | "So flush() was enough. " It was enough against this test. |

ltrace: the same idea, one layer up

strace stops at system calls. ltrace stops at calls into shared libraries.

What it does

It "intercepts and records the dynamic library calls" a process makes [S31]. It can also print "the system calls executed by the program" [S31].

Where it comes from

Juan Cespedes wrote it originally [S32]. The manual page names Petr Machata too [S31]. On Debian and Ubuntu you run `apt-get install ltrace` [S32].

HOW TO USE IT

```
ltrace ./iodemo      # the C library calls: fopen, fwrite, fclose
ltrace -S ./iodemo   # "system calls as well as library calls" [S31]
ltrace -c ./iodemo   # a count for each library call [S31]
```

It runs on Linux only.

[S31] ltrace(1). <https://man7.org/linux/man-pages/man1/ltrace.1.html> [S32] ltrace. <https://ltrace.org/>

Which levels can you actually watch?

I promised three of four boundaries. Here is the tool each needs.

BOUNDARY	WHAT WATCHES IT	IN THIS COURSE
App → library	ltrace sees it, strace does not [V10]	the tool above
Library → kernel	strace — where our tracer attaches [S1]	today
Kernel → device	a tracer for the kernel's block layer [V30]	elsewhere
Device → medium	no system-call trace shows whether the device flushed [V5]	no tracer

The same write, at three levels

Now that the levels have a reason to exist, walk down them with one small program.

The same write, at three levels

LEVEL	PYTHON	C	BUFFERED?
Application	<code>f.write(b"x")</code> [S6]	—	yes [S6]
C library	—	<code>fwrite(...)</code> [S3]	yes [S3]
System call	<code>os.write(...)</code> [S20]	<code>write(...)</code> [S2]	no

The bottom row takes a **file descriptor** — the small integer naming an open file. You saw one already: the `= 3` in the trace line was a file descriptor [S1].

[S1] `strace(1)` — Linux man-pages. <https://man7.org/linux/man-pages/man1/strace.1.html> [S2] `write(2)` — Linux man-pages. <https://man7.org/linux/man-pages/man2/write.2.html> [S3] `fflush(3)` — Linux man-pages. [S6] Python documentation, `open()`. [S20] `os` — Miscellaneous operating system interfaces, `os.write()`. <https://docs.python.org/3/library/os.html#os.write>

Meet the program: `iodemo.c`

Nine lines of C. The first open fails on purpose, so the trace shows an error path.

```
f = fopen("no-such-file.txt", "rb"); /* 1. deliberate failure */
if (f != NULL) { fclose(f); return 2; }

f = fopen(path, "wb");                /* 2. the real file */
if (f == NULL) { return 1; }

fwrite("one\n", 1, 4, f);              /* 3. three writes */
fwrite("two\n", 1, 4, f);              /*    fourteen bytes */
fwrite("three\n", 1, 6, f);

if (fclose(f) != 0) { return 3; }      /* 4. flush, then close */
```

It prints nothing, on purpose. A program that prints nothing adds no write lines of its own to the trace.

Three calls in the source. One in the trace.

3

`fwrite` calls in the source

14

bytes written in total

1

`write` call in the trace

The three `fwrite` calls go into one C library buffer. The manual calls that level "the user-space buffers provided by the C library" [S3]. The line to look for in the live trace, predicted from the source:

```
write(3, "one\ntwo\nthree\n", 14) = 14
```

The 3 is the file descriptor. The 14 is all three `fwrite` calls, in one crossing.

[S3] `fflush(3)` — Linux man-pages. <https://man7.org/linux/man-pages/man3/fflush.3.html>

`fwrite` takes a size and a count

Four arguments, and two of them multiply.

```
fwrite("three\n", 1, 6, f);  
/*      data      ^  ^  stream  
              |  +-- number of items  
              +----- size of one item, in bytes */
```

1 It returns a count

Items written, not bytes. The count equals bytes only when the item size is 1 [S21].

2 Fewer is legal

Below it, "a successful write() may transfer fewer than count bytes" [S2].

[S2] write(2) — Linux man-pages. <https://man7.org/linux/man-pages/man2/write.2.html> [S21] fread(3), which documents fwrite(3) — Linux man-pages. <https://man7.org/linux/man-pages/man3/fwrite.3.html>

Two operating systems, side by side

Read as a comparison. Running the tools is the first lab's job.

Process Monitor: what the kernel did

It reports filesystem and registry activity [S18], which is a different cut than the system-call boundary.

What it does

It shows "real-time file system, Registry and process/thread activity" [S18]. That is a filesystem-and-registry view, not a list of system calls.

Where it comes from

Mark Russinovich wrote the page, in Microsoft's Sysinternals documentation [S18]. The tool "combines the features of two legacy Sysinternals utilities, Filemon and Regmon" [S18]. You download it. You can also run it from Sysinternals Live [S18].

HOW TO USE IT

```
Procmon.exe          # it starts capturing at once
Filter → Process Name is iodemo.exe # keep one process
Operation column     # expect CreateFile and WriteFile rows
```

[S18] Process Monitor. <https://learn.microsoft.com/en-us/sysinternals/downloads/procmon>

NtTrace: a tracer for the Windows Native API

It watches the native API, "the interface between the application space and the OS kernel" [S10].

What it does

It "provides a simple trace facility for the Windows Native API", "roughly equivalent to strace on Linux" [S10]. It attaches the way a debugger does, so it "needs no driver and no administrator rights" [V21].

Where it comes from

Roger Orr wrote it. The source is MIT-licensed, and the release ships pre-built executables [S10]. You run it directly.

HOW TO USE IT

```
NtTrace iodemo.exe          # one line per native call
NtTrace -filter File iodemo.exe # keep calls whose names contain "File" [S10]
NtTrace -only iodemo.exe     # not its child processes [S10]
```

[S10] NtTrace — Roger Orr. <https://rogerorr.github.io/NtTrace/> [S33] NtTrace source. <https://github.com/rogerorr/NtTrace>

Why NtTrace and not Process Monitor

Two Windows tools, two layers. One answers this lecture's question.

PROCESS MONITOR

- 1 Shows "real-time file system, Registry and process/thread activity" [S18]
- 2 The usual Windows choice
- 3 The wrong layer for us

NTTRACE

- 4 Reports native API calls, in user mode [S10]
- 5 Runs on the debugger interface, so it "needs no driver and no administrator rights" [V21]

[S18] Process Monitor — Sysinternals, Microsoft Learn. <https://learn.microsoft.com/en-us/sysinternals/downloads/procmon>

Three Windows words you will need

Win32

The documented interface a Windows program calls. The 32 is history, not a width. Microsoft renamed it the Windows API. The new name "more accurately reflects its roots in 16-bit Windows and its support on 64-bit Windows" [S22].

The native API

The layer below Win32, inside `ntdll.dll`. Microsoft calls it "internal to the operating system" [S9].

The object manager

The kernel component that names open objects. Its paths begin `\\??\\` [V22].

[S9] `winternl.h` — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/api/winternl/nf-winternl-ntquerysysteminformation> [S22]
Windows API index — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/apiindex/windows-api-list>

A real Windows trace

You can already read this. The anatomy matches the Linux line, and the vocabulary differs.

```
NtCreateFile(... ObjectAttributes=0x58:"no-such-file.txt", ...)  
    ⇒ 0xc0000034 [2 'The system cannot find the file specified.']  
NtCreateFile(FileHandle=... [0x124], ... ObjectAttributes=0x58:"iodemo.out", ...) ⇒ 0  
NtWriteFile(FileHandle=0x124, ..., Length=0xe, ...) ⇒ 0  
NtClose(Handle=0x124) ⇒ 0
```

`iodemo.exe` under NtTrace, 2026-09-15 — the same four steps as the Linux trace [S36] [S37]. Ignore every argument except the handle, the file name and the return value.

[S36] Our NtTrace64 capture of `iodemo.exe`, 2026-09-15 — `demo/iodemo-c/NtTrace64-IODEMO-Output.txt` [S37] The container capture of `iodemo`, 2026-09-15 — `demo/iodemo-c/README.md`

A handle is not a file descriptor

First difference, visible in the first argument of almost every line.

LINUX

A file descriptor. A small integer indexing a per-process table. Our trace shows = 3 [S37].

WINDOWS

A handle. An opaque value naming an open object. Our trace shows
FileHandle=0x124 [S36].

Same job in a trace, different mechanism in the system. Do not use one word for the other.

[S36] Our NtTrace64 capture of iodemo.exe, 2026-09-15 — demo/iodemo-c/NtTrace64-IODEMO-Output.txt [S37] The container capture of iodemo, 2026-09-15 — demo/iodemo-c/README.md

Two error namespaces in one return value

Second difference, and it is the layer boundary showing up where nobody expects it.

```
⇒ 0xc0000034 [2 'The system cannot find the file specified.']
```

- | | | |
|----------|----------------------|---|
| 1 | First number | An NTSTATUS, the native layer's code [S36]. |
| 2 | Second number | The Win32 error it translates into [S36]. |
| 3 | Linux | One errno: -1 ENOENT [S37]. |

[S36] Our NtTrace64 capture of iodemo.exe, 2026-09-15 — demo/iodemo-c/NtTrace64-IODEMO-Output.txt [S37] The container capture of iodemo, 2026-09-15 — demo/iodemo-c/README.md

The stable boundary sits at a different level

Third difference, and the other two are symptoms of it.

In our trace of the Windows command shell a path reads "\\??\\C:\\WINDOWS\\system32\\c_852.nls" [S34]. That prefix "is the object-manager namespace, which a Win32 program never types" [V22].

LINUX

[V8]

The system
call interface
is the public
contract.

Syscall

WINDOWS

[S9]

Native entry
points are
"subject to
change from
one release of
Windows to
another".

Win32

[S9] winternl.h — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/api/winternl/nf-winternl-ntquerysysteminformation>

Where the boundary sits is contested history

Two readings survive the evidence. I will not pick one for you.

- 1** **Reading A: deliberate** POSIX, OS/2 and Win32 each reached NT through the native API [S19].
- 2** **Reading B: vestige** A reported POSIX subsystem that was "never really completed" [S15].
- 3** **Linux had no choice** It "had to make the kernel boundary itself the stable interface" [V38].

[S15] The sad history of the Microsoft POSIX subsystem — Reiter (2010). <https://brianreiter.org/2010/08/24/the-sad-history-of-the-microsoft-posix-subsystem/> [S19] Inside Native Applications — Russinovich (2006). <https://learn.microsoft.com/en-us/sysinternals/resources/inside-native-applications>

Linux is the outlier, not Windows

The tempting conclusion is backwards, and the evidence is a list of other systems.

"Linux is the only mainstream system that treats the raw syscall interface as a public contract." [V9]

- | | | |
|----------|----------------------------------|---|
| 1 | Who declines the promise | FreeBSD, OpenBSD, NetBSD, Solaris/illumos, macOS [V43]. Read that list as five silences, not five equally unstable systems [V36]. |
| 2 | Why Windows and macOS can | Both control their own C library [V38]. |
| 3 | So say different | Never say worse. |

One program, two kernels

One program, two systems, traced on one day.

STEP	LINUX [S37]	WINDOWS [S36]
failed open	<code>openat(... "no-such-file.txt" ...) = -1 ENOENT</code>	<code>NtCreateFile(... "no-such-file.txt" ...) ⇒ 0xc0000034 [2 '...']</code>
open	<code>openat(... "iodemo.out" ...) = 3</code>	<code>NtCreateFile(... [0x124] ... "iodemo.out" ...) ⇒ 0</code>
write	<code>write(3, "one\ntwo\nthree\n", 14) = 14</code>	<code>NtWriteFile(FileHandle=0x124, ... Length=0xe ...) ⇒ 0</code>
close	<code>close(3) = 0</code>	<code>NtClose(Handle=0x124) ⇒ 0</code>

Same program, same fourteen bytes, **one write on each side**. Hold the program fixed, and what is left is the two systems.

[S36] Our NtTrace64 capture of `iodemo.exe`, 2026-09-15 — `demo/iodemo-c/NtTrace64-IODEMO-Output.txt` [S37] The container capture of `iodemo`, 2026-09-15 — `demo/iodemo-c/README.md`

What we have not looked at

One question I will not answer, and the work that follows from it.

write() returned success. The bytes are not on a disk.

I built the whole hour towards this sentence. I leave it unresolved.

The manual says it plainly: "A successful return from write() does not make any guarantee that data has been committed to disk" [S2].

Saltzer, Reed and Clark gave the general form in 1984. A lower level may offer "an incomplete version of the function... useful as a performance enhancement" [S11].

So when is it actually safe? That is the durability lecture's question. I leave it open today.

[S2] write(2) — Linux man-pages. [S11] End-to-End Arguments in System Design — Saltzer, Reed and Clark (1984).
<https://web.mit.edu/Saltzer/www/publications/endtoend/endtoend.pdf>

Buffering moves your errors

The error arrives somewhere other than where you caused it.

- 1** **Why it moves** The bytes only reached a buffer.
- 2** **Where it arrives** Where the library flushes, so at `fclose`.
- 3** **The rule** Check the return value of the call that flushes.
- 4** **The idiom hides it** "the most common durability idiom in every mainstream language... contains no durability step anywhere" [V4].

Never retry a failed close

The obvious repair is wrong. Three storage engines dropped the retry in 2018 [V40].

"Retrying the close() after a failure return is the wrong thing to do, since this may cause a reused file descriptor from another thread to be closed" [S4].

The maintainers of three databases reached the same conclusion one level down, for fsync: PostgreSQL, MySQL's InnoDB and MongoDB's WiredTiger [V40]. WiredTiger wrote the reason into its own source comment: "we don't retry fsync-style calls and panic if a flush operation fails" [V24]. Not three separate discoveries, though: that comment cites the LWN report on PostgreSQL [V24].

[S4] close(2) — Linux man-pages. <https://man7.org/linux/man-pages/man2/close.2.html>

Networking and IPC are this same mechanism

Not new subjects. The same objects and the same calls, wearing different names.

- 1 A socket on Linux** It is a file descriptor, and Linux makes that contract binding [V28].
- 2 A socket on Windows** An opaque SOCKET, and Windows makes Winsock the contract [V28].
- 3 A pipe** A file descriptor too, so the same calls reach it [V29].

Depth belongs to the next lecture. I want to stop you filing networking under "something else".

Honest limits

What today's tools cannot see.

- **The device level** [V5]
Not observable from software: no trace can check it.
- **Asynchronous I/O** [V11]
The tracer sees the setup, then no I/O.

Honest limits, continued

And what today's tools cost.

- **strace in production**

[V25]

Even the cheaper `--seccomp-bpf` mode [V39] "should therefore not be used on production systems!"

- **Mapping, crash dumps**

Each has its own lecture.

What to do on Monday

- | | | |
|----------|--|---|
| 1 | Assignment 1 measures
takeaway 1 | Two settings, two patterns, one mebibyte. |
| 2 | Start from section one | Add the random-access variant. |
| 3 | Report your buffer size | Beside every number. |
| 4 | Do not benchmark under the
tracer | Count traced, time untraced [V17]. |
| 5 | Count in the dev container | Both platforms count there. |
| 6 | Week 2 opens | On 22 September. |

Summary — what follows

- | | | |
|---|---------------------------------|---|
| 1 | Ask "accepted or done?" | A fast return reports acceptance, not completion [V12]. |
| 2 | Count, do not reason | No level reports which one holds your data [V16]. |
| 3 | Name the level | Your buffer, the C library, the page cache, the device. |
| 4 | Measure your own machine | Never quote a remembered constant [S7]. |

References • 1/13

- **[S1]** strace(1) — Linux man-pages. <https://man7.org/linux/man-pages/man1/strace.1.html>
- **[S2]** write(2) — Linux man-pages. <https://man7.org/linux/man-pages/man2/write.2.html>
- **[S3]** fflush(3) — Linux man-pages. <https://man7.org/linux/man-pages/man3/fflush.3.html>
- **[S4]** close(2) — Linux man-pages. <https://man7.org/linux/man-pages/man2/close.2.html>
- **[S5]** C99 draft, §7.19.3. <https://busybox.net/~landley/c99-draft.html>
- **[S6]** Built-in Functions — open() — Python documentation.
<https://docs.python.org/3/library/functions.html#open>
- **[S7]** Increase io.DEFAULT_BUFFER_SIZE — CPython issue 117151.
<https://github.com/python/cpython/issues/117151>

References · 2/13

- **[S8]** PEP 3116 — New I/O (2007). <https://peps.python.org/pep-3116/>
- **[S9]** wininternl.h — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/api/winternl/nf-winternl-ntquerysysteminformation>
- **[S10]** NtTrace — Native API tracing for Windows — Roger Orr. <https://rogerorr.github.io/NtTrace/>
- **[S11]** End-to-End Arguments in System Design — Saltzer, Reed and Clark (1984). <https://web.mit.edu/Saltzer/www/publications/endtoend/endtoend.pdf>
- **[S12]** Hints for Computer System Design — Lampson (1983). <https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/acrobat-17.pdf>
- **[S13]** fsync — The Open Group Base Specifications. <https://pubs.opengroup.org/onlinepubs/9699919799/functions/fsync.html>

References · 3/13

- **[S14]** Predict-Observe-Explain — Taber. <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/>
- **[S15]** The sad history of the Microsoft POSIX subsystem — Reiter (2010). <https://brianreiter.org/2010/08/24/the-sad-history-of-the-microsoft-posix-subsystem/>
- **[S16]** FlexSC: Flexible System Call Scheduling with Exception-Less System Calls — Soares and Stumm, OSDI 2010. https://www.usenix.org/legacy/event/osdi10/tech/full_papers/Soares.pdf
- **[S17]** io.IOBase.flush() — Python documentation. <https://docs.python.org/3/library/io.html#io.IOBase.flush>
- **[S18]** Process Monitor — Sysinternals, Microsoft Learn. <https://learn.microsoft.com/en-us/sysinternals/downloads/procmon>

References • 4/13

- **[S19]** Inside Native Applications — Russinovich, Microsoft Learn (2006).
<https://learn.microsoft.com/en-us/sysinternals/resources/inside-native-applications>
- **[S20]** os — Miscellaneous operating system interfaces, os.write() — Python documentation.
<https://docs.python.org/3/library/os.html#os.write>
- **[S21]** fread(3), which documents fwrite(3) — Linux man-pages. <https://man7.org/linux/man-pages/man3/fwrite.3.html>
- **[S22]** Windows API index — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/apiindex/windows-api-list>
- **[S23]** IO_COUNTERS (winnt.h) — Microsoft Learn. https://learn.microsoft.com/en-us/windows/win32/api/winnt/ns-winnt-io_counters

References · 5/13

- **[S24]** GetProcessIoCounters (winbase.h) — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/api/winbase/nf-winbase-getprocessiounters>
- **[S25]** proc_pid_io(5) — Linux man-pages. https://man7.org/linux/man-pages/man5/proc_pid_io.5.html
- **[S26]** Modules/_io/fileio.c — CPython. https://github.com/python/cpython/blob/main/Modules/_io/fileio.c
- **[S27]** Python/fileutils.c — CPython. <https://github.com/python/cpython/blob/main/Python/fileutils.c>
- **[S28]** Low-level I/O — Microsoft Learn, C runtime library. <https://learn.microsoft.com/en-us/cpp/c-runtime-library/low-level-i-o>

References · 6/13

- **[S29]** win32process.GetProcessIoCounters — PyWin32, Mark Hammond.
https://mhammond.github.io/pywin32/win32process__GetProcessIoCounters_meth.html
- **[S30]** strace — project page. <https://strace.io/>
- **[S31]** ltrace(1) — Linux man-pages. <https://man7.org/linux/man-pages/man1/ltrace.1.html>
- **[S32]** ltrace — project page. <https://ltrace.org/>
- **[S33]** NtTrace — source repository, Roger Orr. <https://github.com/rogerorr/NtTrace>
- **[S34]** NtTrace64 capture of cmd.exe /c echo Hello World! — this course (2026-09-15).
<https://ondrasek.github.io/talks/2026-09-15-pb158-lecture-2-io-from-python-to-the-os/demo/NtTrace64-CMD-ECHO-Output.txt>

References · 7/13

- **[S35]** iocount_win32.py, the recorded Windows run — this course (2026-09-15).
<https://ondrasek.github.io/talks/2026-09-15-pb158-lecture-2-io-from-python-to-the-os/demo/iocount-win32/README.md>
- **[S36]** NtTrace64 capture of iodemo.exe — this course (2026-09-15).
<https://ondrasek.github.io/talks/2026-09-15-pb158-lecture-2-io-from-python-to-the-os/demo/iodemo-c/NtTrace64-IODEMO-Output.txt>
- **[S37]** iodemo on Linux and on Windows, the recorded runs — this course (2026-09-15).
<https://ondrasek.github.io/talks/2026-09-15-pb158-lecture-2-io-from-python-to-the-os/demo/iodemo-c/README.md>
- **[V1]** A system call is not a function call — the two-term cost model.
- **[V2]** A system call is not a function call — the measured footprint of one crossing.
- **[V3]** A system call is not a function call — pread and pwrite merge the seek.
- **[V4]** A Successful write() Is a Handoff, Not a Save — the idiom with no durability step.

References · 8/13

- **[V5]** A Successful write() Is a Handoff, Not a Save — the device level is unobservable.
- **[V6]** Standard output is buffered differently depending on where it points — CPython above libc.
- **[V7]** Standard output is buffered differently depending on where it points — the measured 4096-byte buffer.
- **[V8]** Windows and Linux place the stable syscall boundary at different layers — the nomination argument.
- **[V9]** Windows and Linux place the stable syscall boundary at different layers — Linux as the outlier.
- **[V10]** The System Call Boundary — ltrace against strace.
- **[V11]** The System Call Boundary — io_uring with a polling thread.

References · 9/13

- **[V12]** Where the Abstraction Leaks — accepted, not done.
- **[V13]** Where the Abstraction Leaks — trust the counts, not the times.
- **[V14]** Where the Abstraction Leaks — a kill is not a power cut.
- **[V15]** Where the Abstraction Leaks — a privilege transition, not a jump.
- **[V16]** Where the Abstraction Leaks — count, do not deduce.
- **[V17]** Observing a Running System — the 442-times worst case.
- **[V18]** The Memory Hierarchy from Python to Physical Pages — wholesale down, retail up.
- **[V19]** Virtual Memory and the Page Cache — name the cache level in every number.
- **[V20]** File IO and Durability — the device acknowledges early.

References · 10/13

- **[V21]** NtTrace — Native API tracing for Windows (Roger Orr) — a debugger, not a driver.
- **[V22]** NtTrace — Native API tracing for Windows (Roger Orr) — the object-manager path prefix.
- **[V23]** NtTrace — Native API tracing for Windows (Roger Orr) — a trace of a different program is not a matched pair.
- **[V24]** PostgreSQL's fsync() surprise — never retry after fsync fails.
- **[V25]** Every Tracing Mechanism Trades Observation Cost for a Blind Spot — not for production.
- **[V26]** The System Call Boundary — the calls a tracer cannot see.
- **[V27]** A system call is not a function call — what merging the seek did to overall duration.
- **[V28]** Sockets and Network I-O — a Linux socket is a file descriptor, a Windows socket is not.

References · 11/13

- **[V29]** Sockets and Network I-O — sockets, pipes and terminals are all file descriptors.
- **[V30]** File IO and Durability — block-layer tracing watches the kernel-to-device boundary.
- **[V31]** Observing a Running System — the published range for the same benchmark, on other machines.
- **[V32]** Virtual Memory and the Page Cache — the page cache is the machine's own memory.
- **[V33]** File IO and Durability — the kernel issues the device cache-flush command.
- **[V34]** Virtual Memory and the Page Cache — readahead moves data up the same four levels.
- **[V36]** Windows and Linux place the stable syscall boundary at different layers — the five systems do not decline the promise equally.

References · 12/13

- **[V37]** File IO and Durability — FlushFileBuffers is the Windows counterpart of fsync.
- **[V38]** Windows and Linux place the stable syscall boundary at different layers — why Linux could not move its own boundary.
- **[V39]** Every Tracing Mechanism Trades Observation Cost for a Blind Spot — what the cheaper seccomp-bpf mode costs.
- **[V40]** PostgreSQL's fsync() surprise — three engines dropped the retry in 2018.
- **[V41]** PostgreSQL's fsync() surprise — the three engines are independent of each other.
- **[V43]** Windows and Linux place the stable syscall boundary at different layers — the five systems that behave like Windows.
- **[V44]** A system call is not a function call — the 2010 counts should not be quoted as current.

References · 13/13

- **[V45]** A system call is not a function call — the cost model is graded confident, not verified.
- **[V46]** A system call is not a function call — the measured spread in per-crossing cost across hosts.
- **[V47]** NtTrace — Native API tracing for Windows (Roger Orr) — the calls that bypass ntdll are invisible.
- **[V48]** NtTrace — Native API tracing for Windows (Roger Orr) — the debug interface serialises threads.
- **[V49]** NtTrace — Native API tracing for Windows (Roger Orr) — the serialised order is the tracer's, not the program's.

The handout PDF carries the full reference list, and a one-page summary of every [V] source with its own external references.

TAKE IT WITH YOU

Everything from this talk

[Slides \(PDF\)](#)

[Handout \(PDF\)](#)

[Speaker notes \(PDF\)](#)

[Everything \(.zip\) — readable by an AI agent](#)

ondrasek.github.io/talks

me@ondrejkrasicek.com